



首先，恭喜各位能够顺利通过第一个十天，但并不意味着你就一定能进入终极pk。竞赛能成功的都是咬着牙坚持过来的。

其次，竞赛是考察思维的，遇到题多分析，老师讲题时反应要迅速，思维要活跃，这样才能跟上进度，有时一个题要一个小时，甚至更多时间，哪怕中途你只2分钟没跟上，可能后面的一个小时都只能坐火箭了。

最后，纪律是学习氛围的保证。机房是学习的地方，严禁任何同学利用机房电脑打游戏，看无关视频等，一经发现，停止其培训，顶格处理。有问题要讨论的同学也请离开位置到后面3排去小声讨论，避免影响其他同学。



二维数组

int a[6]

1	2	3	4	5	6
---	---	---	---	---	---

一维数组

int a[4][4]

1	2	4	1
2	1	5	2
6	3	7	9
3	5	4	2

a[0][0]	a[0][1]	a[0][2]	a[0][3]
a[1][0]	a[1][1]	a[1][2]	a[1][3]
a[2][0]	a[2][1]	a[2][2]	a[2][3]
a[3][0]	a[3][1]	a[3][2]	a[3][3]

二维数组： 数组名[常量表达1][常量表达2]

int a[4][5]：定义了一个4行5列的整形数组，
下标是a[0][0]—a[3][4].

数组中元素a[2][5]表示？

初始化

一、批量赋值。

$a[2][2] = \{1, 2, 3, 4\};$

1	2
3	4

赋值时**先行后列**。一行赋值完后跳到下一行第一列。

8	7	
5	9	

该怎么赋值？

$a[3][3] = \{8, 7, 0, 5, 9, 0, 0, 0, 0\};$

$a[3][3] = \{\{8, 7\}, \{5, 9\}\};$

二维数组可以看成是由多个一维数组组成的：

$a[3][4]$ 可以看成由3个 $a[4]$ 数组组成或者4个 $a[3]$ 数组组成

初始化

二、循环赋值。

1	2
3	4

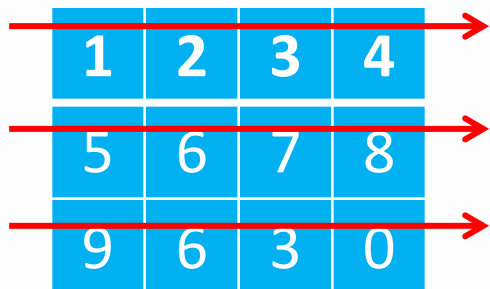
```
for(int i=0;i<=1;i++)
{
    for(int j=0;j<=1;j++)
    {
        scanf("%d",&a[i][j]);
    }
}
```

8	7	
5	9	

```
for(int i=0;i<=1;i++)
{
    for(int j=0;j<=1;j++)
    {
        scanf("%d",&a[i][j]);
    }
}
```

遍历

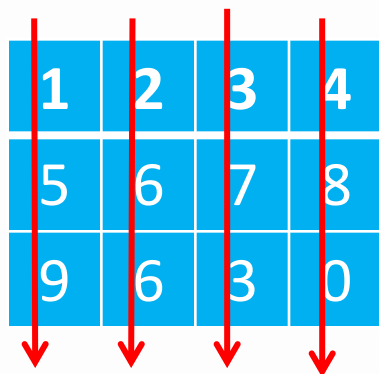
一、按行遍历



A 3x4 grid of blue squares containing the numbers 1 through 8, and 9, 6, 3, 0 in the third row. Three red arrows point horizontally from left to right across each row, indicating the direction of row-major traversal.

1	2	3	4
5	6	7	8
9	6	3	0

二、按列遍历



A 3x4 grid of blue squares containing the numbers 1 through 8, and 9, 6, 3, 0 in the third row. Four red arrows point vertically from top to bottom in each column, indicating the direction of column-major traversal.

1	2	3	4
5	6	7	8
9	6	3	0

```
for(int i=1;i<=3;i++)//行
{
    for(int j=1;j<=4;j++)//列
    {
        scanf("%d",&a[i][j]);
    }
}
for(int j=1;j<=4;j++)//列
{
    for(int i=1;i<=3;i++)//行
    {
        scanf("%d",&a[i][j]);
    }
}
```

无论是哪种遍历方法， $a[i][j]$ 都是表示数组中第*i*行第*j*列的数，这是固定不变的。

字符

输入一个英文字母，并把它输出。

我们以前学习的输入输出格式比如” %d” ,” %f” ,
都是针对数字而言的。但是如果我们需要输入输出
字符呢？

数据类型	定义	输入输出格式	存储空间
整形	int long long	%d %lld	4/8个字节
浮点型	float double	%f %lf	4/8个字节
字符型	char	%c	1个字节

```
char c;  
scanf("%c",&c);  
printf("%c",c);
```

字符

```
int c;  
scanf("%d",&c);  
printf("%d",c);
```

数字

字符输入输出

第一种：

标准输入输出

```
scanf("%c",&c);
```

```
printf("%c",c);
```

和数字一样，
字符使用之前
要先定义

第二种：

字符专用输入输出

```
c=getchar();//输入
```

```
putchar(c);//输出
```

字符专用输入输出
效率更高!!! 但是
只能用于字符的
输入输出。

字符

输入两个字符，输出两个字符，用空格隔开。

输入：x y

```
char a,b;  
scanf("%c%c",&a,&b);  
printf("%c %c",a,b);
```

输出：x y

```
char a,b;  
a=getchar();  
b=getchar();  
putchar(a);  
putchar(b);
```

我们按照要求输入两个字符，输出结果是什么？
如果我们按照这个格式
输出三个字符呢？

字符

当我们输入数的时候，由于数可以是一位，两位，多位，所以当我们输入多个数的时候必须用空格隔开表示我们输入的是多个数。但是字符不一样，**所有的字符都是单个的**，所以当我们输入多个字符的时候是不需要用空格隔开的。同时，**空格和回车**也算是一个字符，所以我们读入的第二个字符是空格，而我们输入的第二个字母没有被读入。

o(空格)k
↓ ↓
变量a 变量b

d(空格)o(空格)g
↓ ↓ ↓
变量a 变量b 变量c

字符

如果输入的字符是**连续输入**:

```
scanf("%c%c",&a,&b);  
printf("%c %c",a,b);
```

如果输入格式**有空格**

```
scanf("%c %c",&a,&b);  
printf("%c %c",a,b);
```

在scanf的%c前面加空格，
会忽略该字符前面的空
字符，包括空格，回车，
而接受后面的有效字符。

如果是字符的输入，输
入写法要和输入格式一
样。以免读入的是空格
或者回车影响结果。

只是和输入格式有关，
和输出格式没有关系，
输出格式和以前一样。

ASCII表

(American Standard Code for Information Interchange 美国标准信息交换代码)

高四位 低四位		ASCII控制字符											ASCII打印字符													
		0000						0001					0010		0011		0100		0101		0110		0111			
		0						1					2		3		4		5		6		7			
		十进制	字符	Ctrl	代码	转义字符	字符解释	十进制	字符	Ctrl	代码	转义字符	字符解释	十进制	字符	十进制	字符	十进制	字符	十进制	字符	十进制	字符	十进制	字符	Ctrl
0000	0	0		^@	NUL	\0	空字符	16	▶	^P	DLE		数据链路转义	32		48	0	64	@	80	P	96	`	112	p	
0001	1	1	☺	^A	SOH		标题开始	17	◀	^Q	DC1		设备控制 1	33	!	49	1	65	A	81	Q	97	a	113	q	
0010	2	2	☹	^B	STX		正文开始	18	↕	^R	DC2		设备控制 2	34	"	50	2	66	B	82	R	98	b	114	r	
0011	3	3	♥	^C	ETX		正文结束	19	!!	^S	DC3		设备控制 3	35	#	51	3	67	C	83	S	99	c	115	s	
0100	4	4	♦	^D	EOT		传输结束	20	¶	^T	DC4		设备控制 4	36	\$	52	4	68	D	84	T	100	d	116	t	
0101	5	5	♣	^E	ENQ		查询	21	§	^U	NAK		否定应答	37	%	53	5	69	E	85	U	101	e	117	u	
0110	6	6	♠	^F	ACK		肯定应答	22	—	^V	SYN		同步空闲	38	&	54	6	70	F	86	V	102	f	118	v	
0111	7	7	●	^G	BEL	\a	响铃	23	↕	^W	ETB		传输块结束	39	'	55	7	71	G	87	W	103	g	119	w	
1000	8	8	◼	^H	BS	\b	退格	24	↑	^X	CAN		取消	40	(	56	8	72	H	88	X	104	h	120	x	
1001	9	9	◯	^I	HT	\t	横向制表	25	↓	^Y	EM		介质结束	41	)	57	9	73	I	89	Y	105	i	121	y	
1010	A	10	◐	^J	LF	\n	换行	26	→	^Z	SUB		替代	42	*	58	:	74	J	90	Z	106	j	122	z	
1011	B	11	♂	^K	VT	\v	纵向制表	27	←	^[	ESC	\e	溢出	43	+	59	;	75	K	91	[	107	k	123	{	
1100	C	12	♀	^L	FF	\f	换页	28	└	^\	FS		文件分隔符	44	,	60	<	76	L	92	\	108	l	124		
1101	D	13	♪	^M	CR	\r	回车	29	↔	^]	GS		组分分隔符	45	-	61	=	77	M	93	]	109	m	125	}	
1110	E	14	🎵	^N	SO		移出	30	▲	^^	RS		记录分隔符	46	.	62	>	78	N	94	^	110	n	126	~	
1111	F	15	🎵	^O	SI		移入	31	▼	^.	US		单元分隔符	47	/	63	?	79	O	95	_	111	o	127	␣	^Backspace 代码: DEL

注：表中的ASCII字符可以用“Alt + 小键盘上的数字键”方法输入。

2013/08/08

ASCII

ASCII 0表示字符中的空字符' \0' , ' \0' 通常用来表示字符串的结束。

' ' (空格)和' \n' (回车)也表示字符

字符中也有' 0' —' 9' , 但是字符中的' 0' —' 9' 对应的ASCII值是48--57

字符中的大小写字母对应的ASCII值是不一样的, 所以字母是分大小写的。大写字母表和小写字母表在ASCII中是分开的。

键盘上所有对应的单个符号都是字符。

ASCII和字符的转换

问题一：如果我要知道某个字符对应的ASCII怎么办？

```
char a;  
scanf("%c",&a);    printf("%d",a);
```

如果输入字符，当输出格式是%c时表示输出字符，如果输出格式是%d表示输出该字符对应的ASCII值。

问题二：我怎么知道输入的数字是字符还是数字？

```
char a;                int a;  
scanf("%c",&a);        scanf("%d",&a);  
char a='1';           int a=1;
```

接收字符时不能和整数转换。而在输出，或运算时，字符和ASCII码可相互转换。

简单计算

输入一个表达式：形如a(空格)+(空格)b。
(a, b是数字) 输出表达式的结果，表达式符号
可以为“+”，“-”，“*”，“/”，“%”。

输入：

2 + 3

输出：

5

输入：

5 / 3

输出：

1

想一想：怎么判断表达式符号？

源代码

```
#include<stdio>
using namespace std;
int main()
{
```

因为中间有字符的输入，所以一定要注意格式，如果没有第一个空格，会把空格读入变量c

```
    int a,b;    char c;
    scanf("%d %c %d",&a,&c,&b);
    if(c=='+')   printf("%d",a+b);
    else if(c=='-') printf("%d",a-b);
    else if(c=='*') printf("%d",a*b);
    else if(c=='/') printf("%d",a/b);
    else if(c=='%') printf("%d",a%b);
    return 0;
```

```
}
```

注意字符要用‘ ’括起来

数字分离

输入一个六位数，分别输出它的每一位，中间用空格隔开

输入：

123456

输出：

1 2 3 4 5 6

在没学字符之前是怎么做的？

```
char a,b,c,d,e,f;  
scanf("%c%c%c%c%c%c",&a,&b,&c,&d,&e,&f);  
printf("%c %c %c %c %c %c",a,b,c,d,e,f);
```

想一想：怎么把数字1和
字符1相互转换

打印字母表

输入n，第一行正序打印前n个小写字母，第二行倒叙打印后n个大写字母。

输入：
5

输出：
a b c d e
Z Y X W V

想一想：字符中相邻两个字母之间有什么关系？

源代码

```
#include<cstdio>
using namespace std;
int main()
{
    int n;
    scanf("%d",&n);
    for(int i=1;i<=n;i++)
        printf("%c ", 'a'+i-1);
    printf("\n");
    for(int i=1;i<=n;i++)
        printf("%c ", 'Z'-i+1);
    return 0;
}
```

‘a’+1表示什么意思？

当字符和字符或者字符和数字进行计算的时候，字符会转换成对应的ASCII来计算，至于要输出数字还是字符由输出格式决定

也可以写成：

```
for(int i='a';i<='a'+n-1;i++)
    printf("%c ",i);
```

字符数组

存储多个字符的数组被称为字符数组，也可以称为字符串。

定义：char s[100];

输入：

第一种：

```
scanf("%s", s);
```

第二种：

```
for (int i=1; i<=n; i++)  
scanf("%c", &s[i]);
```

区别：

- 1: 字符串的整体输入用%s，并且输入不用"&"，而字符的单个输入要用"&"。
- 2: 字符串可以整体赋值也可以单个赋值，单个赋值用%c,循环赋值
- 3: %s输入字符串的下标是从0开始。如果不知道字符串到底有长，只能用第一种输入。

输入

第一种：

```
scanf ("%s", s);
```

第二种：

```
for (int i=1; i<=n; i++)  
scanf ("%c", &s[i]);
```

区别：

- 1：字符串的整体输入用%s，并且输入不用“&”，直接输入数组名而不需要数组下标，表示整个数组。而字符的单个输入要用“&”，需要数组下标，表示单个元素。
- 2：字符串可以整体赋值也可以单个赋值，单个赋值用%c表示单个元素，使用循环赋值。
- 3：%s输入字符串的下标只能是从0开始。
- 4：如果不知道字符串到底有多长，只能用第一种输入。
- 5：如果字符串中含有空格，只能用第二种输入。

输出

第一种：

```
printf("%s", s);
```

第二种：

```
for (int i=1; i<=n; i++)  
printf("%c", &s[i]);
```

输出格式和输入一样，最好输入输出格式匹配，以免因为下标不一样造成不必要的麻烦。

一：第一种输出仍然是从下标为0的元素开始输出。

为什么第一种输入不能包括空格：

因为scanf函数接收字符串，当遇到空格或者回车时表示结束。

```
scanf("%s", s);
```

如果输入hello world，实际上读入数组s的只有hello，空格表示读入结束

数组赋值

```
char s[100]={'h','e','l','l','o'}; ✓
```

```
char s[100]= "hello"; ✓
```

```
char s[100];  
s="hello"; ✗
```

字符用单引号，字符数组用双引号。字符串定义之后不能直接整体赋值。

```
char s[100];  
s[0]='h';  
s[1]='e';  
s[2]='l';  
s[3]='l';  
s[4]='o'; ✓
```

字符串输出

输入一个英文句子，输并把它输出。

输入：

hello world

`scanf("%s",s);` ❌

输出：

hello world

`scanf()` 函数遇到空格表示一次输入结束。

`printf()` 函数只有遇到 ‘\0’ 才表示输出结束。

如果遇到带空格的句子，输入不能用%s，只能用%c一个一个读入，当读入回车的时候表示读入结束。这种情况下，一定要注意最好让字符串的下标从0开始，以免后面的函数失效。

注意事项

1. 字符串定义的时候初始值都是 ‘\0’ , ‘\0’ 表示空字符, 同时也表示字符串的结尾。

‘\0’	‘\0’	‘\0’	‘\0’	‘\0’	‘\0’	‘\0’	‘\0’
------	------	------	------	------	------	------	------

2. 如果我是手动赋值, 如果不从s[0]开始, 当使用%s时输出无结果。

```
for(int i=1;i<=n;i++)    scanf("%c",s[i]);  
printf("%s",s);           无输出  
for(int i=1;i<=n;i++)    printf("%c",s[i]);
```

‘\0’	h	e	l	l	o	‘\0’	‘\0’
s[0]	s[1]	s[2]	s[3]	s[4]	s[5]	s[6]	s[6]

所有字符串函数读到 ‘\0’ 结束

字符串函数

复制函数：

strcpy(字符串名1, 字符串名2)：

将字符串2**完全**复制给字符串1.

```
char a[100], b[100];
```

```
strcpy(a,b);
```

strncpy(字符串名1, 字符串名2, 长度n)：

将字符串2中前n个字符复制给字符串1.

```
char a[100], b[100];
```

```
strncpy(a,b,5);
```

字符串函数

比较函数：

strcmp(字符串名1, 字符串名2)：

比较两个字符串的大小(ASCII)，从第一个字符开始比较，如果字符串1>字符串2, 结果返回正整数，相等则返回0，小于则返回负整数。

```
char a[100]="apple";   char b[100]="Apple";  
if(strcmp(a,b)>0)      ...  
else if(strcmp(a,b)=0) ...  
else if(strcpy(a,b)<0)  ...
```

strncmp(字符串名1, 字符串名2, n)：

比较两个字符串前n个字符的大小关系。返回值和上面一样

字符串函数

连结函数：

strcat(字符串名1, 字符串名2)：

把字符串2连接到字符串1的后面，组成一个新的字符串1(注意是字符串1发生改变).

```
char a[10]="chi";  
char b[10]="na";  
strcat(a,b);
```

strncat(字符串名1, 字符串名2, n)：

把字符串2前那个字符连接到字符串1的后面，组成一个新的字符串1.

注意字符串连接时，a字符串的长度至少应该为？

字符串函数

其他函数：

strlen(字符串名)：

计算字符串的长度，返回值为字符串的长度，需要用一个变量来保存

```
printf("%s",a);  l=strlen(a);
```

strlwr(字符串名)：

把字符串中所有大写字母转换成小写字母。

strupr(字符串名)：

把字符串中所有小写字母转换成大写字母。

**注意：所有的字符串函数的使用都需要
新增头文件：#include<cstring>**

N 位数反转

输入一行数，反向输出。去除前导零。

输入：
123456

输出：
654321

输入：
1000000

输出：
1

输入数据 ≤ 1000 位。



数字之和

输入一行数，求出每一位数之和。

输入：

1234

输出：

10

输入：

10000

输出：

1

